



SWAPI Grid

Star Wars Fleet — Documentation technique complète du projet

Angular 21

TypeScript 5.9

AG Grid Community 36

Tailwind CSS 4

RxJS 7.8

Vitest 4

Application front-end seule consommant l'API publique SWAPI

Déployée sur Netlify — <https://swapi-grid.netlify.app>

Document généré le 15 juillet 2026

Sommaire

1. Vue d'ensemble du projet

- Objectif fonctionnel
- La stack technique et pourquoi ces choix

2. Structure des dossiers et fichiers

- Arborescence commentée
- Rôle de chaque fichier de configuration

3. Le démarrage de l'application (bootstrap)

- index.html → main.ts → App → StarshipGridComponent
- La configuration zoneless

4. Le service SwapiService

- Les interfaces TypeScript
- Le cache de pages
- Les 4 méthodes publiques en détail

5. Le composant StarshipGridComponent

- L'état avec les signals
- La définition des colonnes
- Le datasource et l'infinite scroll
- Le tri personnalisé
- La recherche avec debounce
- L'édition de cellule

6. Le template HTML

- Structure générale
- Le loader, l'erreur, le footer
- Les options passées à ag-grid-angular

7. Les styles

- styles.css (global) et le @theme Tailwind v4
- starship-grid.css et ViewEncapsulation.None
- Les icônes Material dans les en-têtes

8. Les tests unitaires

- swapi.service.spec.ts
- starship-grid.spec.ts

9. Le déploiement Netlify

10. Concepts clés et pièges à connaître

1. Vue d'ensemble du projet

1.1 Objectif fonctionnel

SWAPI Grid ("Star Wars Fleet") est une **single-page application 100 % front-end** qui affiche la liste des vaisseaux spatiaux de l'univers Star Wars dans un tableau de données interactif. Les données proviennent de l'API publique gratuite **SWAPI** (<https://swapi.dev/api/starships/>), qui est en lecture seule et paginée par blocs de 10 résultats.

Les fonctionnalités de l'application :

- **Scroll infini** : les pages de l'API sont chargées au fur et à mesure du défilement, sans bouton "page suivante" et *sans loader visible* pendant le scroll (exigence du cahier des charges).
- **Recherche par nom** : un champ de recherche filtre les vaisseaux uniquement sur le champ `name`, avec affichage du nombre de résultats.
- **Tri des colonnes** : tri numérique pour les colonnes chiffrées (Crew, Passengers...), tri alphabétique naturel pour les autres, valeurs "unknown" toujours en fin de liste.
- **Édition d'une colonne** : la colonne `cargo_capacity` est éditable ; la modification vit uniquement côté client (SWAPI est en lecture seule).
- **Gestion d'erreur** : si l'API échoue, un message et un bouton Retry apparaissent.
- **Numérotation des lignes**, bordures de colonnes, icônes d'en-tête, police Inter, footer de fin de liste.

1.2 La stack technique et pourquoi ces choix

Technologie	Rôle	Points notables dans ce projet
Angular 21	Framework applicatif	Composants <i>standalone</i> (pas de NgModule), détection de changement zoneless (sans zone.js), état géré par signals .
AG Grid Community 36	La grille de données	Utilisée avec le Infinite Row Model : la grille demande les lignes bloc par bloc à un "datasource" que l'on écrit soi-même.
Tailwind CSS 4	Tout le style applicatif	Version 4 = configuration <i>CSS-first</i> (directive <code>@theme</code> dans le CSS, plus de <code>tailwind.config.js</code>). Branché via PostCSS (<code>.postcssrc.json</code>).
RxJS 7.8	Asynchrone / flux	Debounce de la recherche, cache d'observables partagés (<code>shareReplay</code>), combinaison de pages (<code>forkJoin</code>).
Vitest 4	Tests unitaires	Runner de test officiel du builder Angular (<code>ng test</code>), environnement jsdom.
@fontsource/inter material-icons	Police + icônes	Auto-hébergées via npm — <i>aucun CDN externe au runtime</i> , tout est empaqueté dans le build.

À savoir — Il n'y a **aucun backend** dans ce projet. Le seul serveur impliqué est SWAPI (tiers, public). Le site est hébergé statiquement sur Netlify.

2. Structure des dossiers et fichiers

2.1 Arborescence commentée

```
swapi-grid/ └─ src/ │ └─ index.html <!-- squelette HTML, monte <app-root> --> │ └─ main.ts <!-- point d'entrée : bootstrapApplication --> │ └─ styles.css <!-- CSS global : imports Tailwind/AG Grid/polices + @theme --> │ └─ app/ │ └─ app.ts <!-- composant racine App (coquille minimale) --> │ └─ app.html <!-- rend uniquement <app-starship-grid /> --> │ └─ app.config.ts <!-- providers globaux (HttpClient, error listeners) --> │ └─ app.spec.ts <!-- test du composant racine --> │ └─ core/ │ │ └─ services/ │ │ │ └─ swapi.service.ts <!-- accès API + cache + interfaces --> │ │ │ └─ swapi.service.spec.ts <!-- tests du service --> │ │ │ └─ features/ │ │ │ └─ starship-grid/ │ │ │ │ └─ starship-grid.ts <!-- LE composant principal (grille) --> │ │ │ │ └─ starship-grid.html <!-- template : header, recherche, grille, footer --> │ │ │ │ └─ starship-grid.css <!-- overrides CSS ciblant le DOM interne d'AG Grid --> │ │ │ │ └─ starship-grid.spec.ts <!-- tests du composant (tri, édition) --> │ │ │ └─ public/ │ │ │ └─ favicon.ico │ │ │ └─ _redirects <!-- règle SPA Netlify : /* → /index.html 200 --> │ │ │ └─ angular.json <!-- config du build Angular (esbuild) --> │ │ │ └─ package.json <!-- dépendances + scripts npm --> │ │ │ └─ .postcssrc.json <!-- branche le plugin PostCSS de Tailwind v4 --> │ │ │ └─ netlify.toml <!-- commande de build + dossier publié --> │ │ │ └─ .nvmrc <!-- version de Node épinglée pour Netlify --> │ │ │ └─ tsconfig.json / .app.json / .spec.json <!-- configs TypeScript --> │ │ │ └─ .prettierrc / .editorconfig <!-- formatage --> │ │ │ └─ README.md <!-- réponses aux points du cahier des charges -->
```

La structure suit une convention Angular classique : `core/` regroupe ce qui est transversal (les services), `features/` regroupe les fonctionnalités métier (ici une seule : la grille de vaisseaux). Chaque fonctionnalité rassemble ses 4 fichiers (TS, HTML, CSS, spec) dans son propre dossier.

À savoir — Le projet utilise la convention de nommage **Angular 21 sans suffixe** : les fichiers s'appellent `starship-grid.ts` et non `starship-grid.component.ts`. En revanche, la classe exportée garde le suffixe explicite : `StarshipGridComponent`.

2.2 Rôle de chaque fichier de configuration

package.json — scripts et dépendances

Script	Commande	Usage
<code>npm start</code>	<code>ng serve</code>	Serveur de dev (rechargement à chaud) sur <code>http://localhost:4200</code>
<code>npm run build</code>	<code>ng build</code>	Build de production dans <code>dist/swapi-grid/browser</code>
<code>npm test</code>	<code>ng test</code>	Lance les tests unitaires avec Vitest

angular.json — le build

- `"builder": "@angular/build:application"` : le builder moderne basé sur **esbuild + Vite** (rapide, remplace Webpack).
- `"styles": ["src/styles.css"]` : l'unique feuille de style *globale* compilée par le build — c'est important pour Tailwind (voir chapitre 7).
- `"assets": [{"glob": "**/*", "input": "public"}]` : tout le dossier `public/` est copié tel quel dans le build (favicon, `_redirects`).
- Budgets de taille en production : warning à 2 MB, erreur à 3 MB pour le bundle initial.

.postcssrc.json — le pont vers Tailwind

```
{
  "plugins": {
    "@tailwindcss/postcss": {}
  }
}
```

⚠ **Piège** — Le builder Angular lit **uniquement** `.postcssrc.json` — un fichier `postcss.config.js` serait totalement ignoré (cela a été vérifié empiriquement durant le développement, et un doublon `postcss.config.js` a d'ailleurs été supprimé du projet). Si Tailwind ne s'applique plus, c'est le premier fichier à vérifier.

tsconfig.json — TypeScript strict

Le mode `strict` est activé, plus des options de rigueur supplémentaires (`noImplicitReturns` , `noFallthroughCasesInSwitch` , `strictTemplates` pour les templates Angular...). Concrètement : null-safety obligatoire, tout est typé, y compris les bindings dans le HTML.

netlify.toml + public/_redirects + .nvmrc

```
# netlify.toml
[build]
  command = "npm run build"
  publish = "dist/swapi-grid/browser"
```

- `_redirects` contient `/* /index.html 200` : toute URL est réécrite vers `index.html` (indispensable pour une SPA, sinon un rafraîchissement sur une sous-route donnerait un 404).
- `.nvmrc` épingle la version de Node que Netlify doit utiliser pour builder, afin d'éviter les écarts entre build local et build CI.

3. Le démarrage de l'application

3.1 La chaîne de bootstrap

Au chargement de la page, la chaîne est : **index.html** → **main.ts** → **App** → **StarshipGridComponent**.

src/index.html

```
<body>
  <app-root></app-root>
</body>
```

Le HTML servi au navigateur ne contient que la balise `<app-root>`. Angular injecte les scripts au build.

src/main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { App } from './app/app';

bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

`bootstrapApplication` est l'API de démarrage **standalone** d'Angular moderne : pas de `AppModule`, on démarre directement un composant avec sa configuration.

src/app/app.config.ts

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideHttpClient()
  ]
};
```

- `provideHttpClient()` enregistre le client HTTP d'Angular — sans lui, l'injection de `HttpClient` dans `SwapiService` échouerait.
- `provideBrowserGlobalErrorListeners()` capte les erreurs globales non attrapées du navigateur et les route vers le gestionnaire d'erreurs d'Angular.

src/app/app.ts + app.html

```
@Component({
  selector: 'app-root',
  imports: [StarshipGridComponent],
  templateUrl: './app.html'
})
export class App {}

<!-- app.html -->
<app-starship-grid />
```

Le composant racine est une **coquille vide volontaire** : son seul rôle est de rendre la fonctionnalité principale. Toute la logique vit dans `StarshipGridComponent`. Il n'y a pas de routeur : l'application n'a qu'un seul écran (le package `@angular/router` a été retiré des dépendances).

3.2 La détection de changement zoneless

⚠ **Piège** — Ce projet tourne **sans zone.js** (mode "zoneless" d'Angular 21). Conséquence majeure : Angular ne re-rend *pas* automatiquement le template quand on modifie une propriété de classe ordinaire depuis un callback asynchrone (ex. : un `subscribe` RxJS). C'est pourquoi **tout l'état affiché passe par des signals** (`signal()`) : écrire dans un signal (`monSignal.set(...)`) notifie Angular qu'il faut re-rendre. Si un jour une valeur affichée "ne se met pas à jour", le réflexe est de vérifier qu'elle est bien un signal et non une simple propriété.

4. Le service SwapiService

`src/app/core/services/swapi.service.ts` est la **seule porte d'entrée vers l'API**. Le composant ne fait jamais de HTTP directement : il demande au service, qui gère l'URL, le cache et la combinaison des pages.

4.1 Les interfaces TypeScript

```
export interface Starship {
  name: string;
  model: string;
  manufacturer: string;
  cost_in_credits: string;
  length: string;
  max_atmosphering_speed: string;
  crew: string;          // Δ chaîne brute : "30-165", "47,060", "unknown"
  passengers: string;
  cargo_capacity: string;
  consumables: string;
  hyperdrive_rating: string;
  MGLT: string;
  starship_class: string;
  pilots: string[];      // URLs des pilotes connus
  films: string[];
  created: string;
  edited: string;
  url: string;
}

export interface SwapiResponse {
  count: number;          // nombre TOTAL de vaisseaux (toutes pages)
  next: string | null;    // URL de la page suivante (null si dernière)
  previous: string | null;
  results: Starship[];    // les 10 vaisseaux de la page courante
}
```

À savoir — SWAPI renvoie **tous les champs numériques sous forme de chaînes**, parfois exotiques (`"30-165"` pour une fourchette, `"1,000,000"` avec séparateurs de milliers, `"unknown"` quand la donnée n'existe pas). C'est la raison d'être du parsing numérique du chapitre 5.4.

4.2 Le cache de pages — le cœur du service

```
private readonly cache = new Map<string, Observable<SwapiResponse>>();

getStarships(page: number = 1): Observable<SwapiResponse> {
  const key = `${page}`;
  const cached = this.cache.get(key);
  if (cached) {
    return cached; // ① déjà demandé → même observable
  }

  const request$ = this.http.get<SwapiResponse>(this.baseUrl, { params: { page } }).pipe(
    shareReplay(1), // ② rejoue la réponse aux abonnés suivants
    catchError((error) => {
      this.cache.delete(key); // ③ échec → on retire l'entrée du cache
      return throwError(() => error);
    })
  );

  this.cache.set(key, request$);
  return request$;
}
```

Trois mécanismes travaillent ensemble :

1. **La Map de cache** associe chaque numéro de page à l'observable de sa requête. On met en cache *l'observable lui-même*, pas la réponse : même deux appels quasi simultanés à la même page ne déclenchent qu'une seule requête HTTP.
2. `shareReplay(1)` mémorise la dernière valeur émise et la rejoue instantanément à tout nouvel abonné. Résultat : re-scroller vers des lignes déjà vues ne refait jamais d'appel réseau.
3. `catchError` + **évacuation** : sans le `cache.delete`, une requête échouée resterait en cache et rejouerait *l'erreur* pour toujours — le bouton Retry ne servirait à rien. En évacuant l'entrée, le retry relance une vraie requête.

4.3 Les autres méthodes

getAllStarships() — combiner toutes les pages

```
getAllStarships(): Observable<Starship[]> {
  return this.getStarships(1).pipe(
    switchMap((first) => {
      const totalPages = Math.ceil(first.count / first.results.length);
      const remainingPages = Array.from({ length: totalPages - 1 }, (_, i) => this.getStarships(i + 2));

      return remainingPages.length
        ? forkJoin(remainingPages).pipe(
            map((pages) => [...first.results, ...pages.flatMap((page) => page.results)])
          )
        : of(first.results);
    })
  );
}
```

Logique en deux temps :

1. On charge la **page 1**, qui contient `count` (le total). On en déduit le nombre de pages : `ceil(36 / 10) = 4`.
2. `forkJoin` lance les pages 2 à N **en parallèle** et attend qu'elles soient toutes arrivées, puis `flatMap` aplatit le tout en un seul tableau ordonné.

Comme chaque page passe par `getStarships()` (donc par le cache), un second appel à `getAllStarships()` est quasi instantané et ne refait aucun appel réseau.

searchStarshipsByName() — la recherche

```
// SWAPI's `search` param also matches the `model` field, so it can't be used
// to search by name only. Instead, fetch every (cached) page and filter locally.
searchStarshipsByName(name: string): Observable<Starship[]> {
  const query = name.toLowerCase();
  return this.getAllStarships().pipe(
    map((starships) => starships.filter((s) => s.name.toLowerCase().includes(query)))
  );
}
```

⚠ **Piège** — SWAPI possède bien un paramètre `?search=`, mais il a été écarté *volontairement* : il matche aussi le champ `model`, alors que l'exigence était de chercher **uniquement sur le nom**. La recherche récupère donc tout le jeu de données (36 vaisseaux, mis en cache) et filtre localement, insensible à la casse, en sous-chaîne.

updateStarship() — l'édition simulée

```
updateStarship(starship: Starship, changes: Partial<Starship>): Observable<Starship> {
  // SWAPI is read-only. Once a real backend supports writes, swap this for:
  // return this.http.patch<Starship>(starship.url, changes);
  return of({ ...starship, ...changes });
}
```

SWAPI ne permet aucune écriture. La méthode simule un succès en renvoyant l'objet fusionné. L'intérêt de garder cette indirection : le composant est déjà câblé comme si un vrai backend existait (avec rollback en cas d'erreur — voir 5.6) ; le jour où un backend accepte les écritures, une seule ligne change.

5. Le composant StarshipGridComponent

`src/app/features/starship-grid/starship-grid.ts` (~240 lignes) est le cerveau de l'application. Il orchestre AG Grid, le service, la recherche, le tri et l'édition.

5.1 Le décorateur et l'encapsulation

```
ModuleRegistry.registerModules([AllCommunityModule]); // active toutes les features Community d'AG Grid

@Component({
  selector: 'app-starship-grid',
  imports: [AgGridAngular],
  templateUrl: './starship-grid.html',
  styleUrls: ['./starship-grid.css'],
  // This stylesheet targets AG Grid's internally-rendered DOM (not part of
  // Angular's own template) and a Tailwind @theme override meant to apply
  // globally, so it must not be scoped to this component.
  encapsulation: ViewEncapsulation.None,
})
export class StarshipGridComponent { ... }
```

- `ModuleRegistry.registerModules([AllCommunityModule])` est exigé par AG Grid v36 : sans cet enregistrement, la grille refuse de fonctionner (architecture modulaire).
- `ViewEncapsulation.None` désactive l'isolation de style du composant. Nécessaire car `starship-grid.css` doit atteindre le DOM *général* par AG Grid (en-têtes, icônes de tri), qui ne porte pas les attributs de scoping qu'Angular ajoute aux éléments de son propre template.

5.2 L'état — 5 signals + injections

```
private readonly swapiService = inject(SwapiService); // injection moderne par fonction
private readonly searchInput$ = new Subject<string>(); // flux brut des frappes clavier
private gridApi?: GridApi; // poignée vers l'API d'AG Grid

searchTerm = signal(''); // terme de recherche actif (après debounce)
isLoading = signal(true); // true UNIQUEMENT pendant le tout premier chargement
hasError = signal(false); // affiche l'écran d'erreur + Retry
totalCount = signal<number | null>(null); // total de résultats (sert au compteur et au footer)
reachedEnd = signal(false); // true quand la dernière ligne est visible → footer
```

Signal	Écrit par	Lu par (template)
<code>searchTerm</code>	<code>onSearch()</code> (après debounce 300 ms)	condition d'affichage du compteur de résultats
<code>isLoading</code>	le datasource (premier bloc uniquement)	overlay spinner
<code>hasError</code>	datasource (erreur) / <code>onRetry()</code>	écran d'erreur + bouton Retry
<code>totalCount</code>	datasource (chaque réponse)	"X results found" + footer
<code>reachedEnd</code>	<code>onBodyScrollEnd()</code>	affichage du footer "End of list"

5.3 La définition des colonnes

```
defaultColDef: ColDef = {
  cellClass: '!border-r !border-gray-100',      // bordure droite de chaque cellule (Tailwind)
  headerClass: 'border-r border-gray-200',      // bordure droite de chaque en-tête
  sortable: true,                               // toutes les colonnes triables par défaut
};

colDefs: ColDef[] = [
  {
    colId: 'rowNumber',
    headerName: '#',
    valueGetter: (params) => (params.node?.rowIndex ?? 0) + 1, // numéro de ligne calculé
    pinned: 'left', width: 60,
    resizable: false, sortable: false, filter: false,
  },
  { field: 'name', headerName: 'Name', resizable: true, headerClass: '... hdr-icon-name' },
  { field: 'starship_class', headerName: 'Class', resizable: true, headerClass: '... hdr-icon-class' },
  { field: 'model', headerName: 'Model', resizable: true, headerClass: '... hdr-icon-model' },
  { field: 'manufacturer', headerName: 'Manufacturer', resizable: true, headerClass: '... hdr-icon-manufacturer' },
  { field: 'pilots.length', headerName: 'Known Pilots', resizable: false, headerClass: '... hdr-icon-pilots' },
  { field: 'crew', headerName: 'Crew', resizable: false, headerClass: '... hdr-icon-crew' },
  { field: 'passengers', headerName: 'Passengers', resizable: false, headerClass: '... hdr-icon-passengers' },
  { field: 'cargo_capacity', headerName: 'cargo capacity', resizable: false, editable: true,
    headerClass: '... hdr-icon-cargo' },
];
```

Points à comprendre :

- **La colonne #** n'existe pas dans les données : son `valueGetter` calcule `rowIndex + 1` à la volée. Elle est épinglée à gauche (`pinned: 'left'`), non triable et non redimensionnable. *Pourquoi pas l'option native `rowNumbers` d'AG Grid ?* Parce qu'elle est réservée à la version Enterprise (vérifié empiriquement — erreur #200).
- `field: 'pilots.length'` : AG Grid interprète le point comme un chemin profond. On affiche donc directement le *nombre* de pilotes du tableau `pilots`.
- `editable: true` uniquement sur `cargo_capacity` : double-cliquer sur une cellule de cette colonne ouvre l'éditeur intégré.
- Les classes `hdr-icon-*` sont des marqueurs CSS pour injecter les icônes Material dans les en-têtes (voir chapitre 7.3).
- Le préfixe Tailwind `!` dans `!border-r` génère du `!important` : nécessaire car AG Grid pose lui-même un `border: 1px solid transparent` sur `.ag-cell` qui gagnerait sinon.

5.4 Le datasource — le cœur de l'infinite scroll

Avec `rowModelType: 'infinite'`, AG Grid ne reçoit pas un tableau de données : il appelle `dataSource.getRows(params)` chaque fois qu'il a besoin d'un bloc de lignes (au chargement, puis au fil du scroll). `params` contient `startRow`, `endRow`, le modèle de tri, et deux callbacks (`successCallback` / `failCallback`).

```

dataSource: IDatasource = {
  getRows: (params: IGetRowsParams<Starship>) => {
    const isFirstBlock = params.startRow === 0;

    if (isFirstBlock) {
      this.isLoading.set(true);          // loader UNIQUEMENT pour le premier bloc
    }

    const search = this.searchTerm();
    const sortModel = params.sortModel;

    const request$ =
      search || sortModel.length
      // CAS A – recherche et/ou tri actif : on travaille sur le jeu complet
      ? (search ? this.swapiService.searchStarshipsByName(search)
        : this.swapiService.getAllStarships()).pipe(
        map((results) => {
          const sorted = this.applySort(results, sortModel);
          return {
            results: sorted.slice(params.startRow, params.endRow), // découpe le bloc demandé
            count: sorted.length,
          };
        }),
      )
      // CAS B – ni recherche ni tri : 1 bloc AG Grid = 1 page SWAPI
      : this.swapiService.getStarships(Math.floor(params.startRow / PAGE_SIZE) + 1);

    request$
      .pipe(finalize(() => { if (isFirstBlock) this.isLoading.set(false); }))
      .subscribe({
        next: (response) => {
          if (isFirstBlock) this.hasError.set(false);
          this.totalCount.set(response.count);
          params.successCallback(response.results, response.count); // livre les lignes à AG Grid
        },
        error: () => {
          if (isFirstBlock) this.hasError.set(true);
          params.failCallback();
        },
      });
  },
};

```

Décomposons les décisions de design :

- **"Pas de loader pendant le scroll"** (exigence du cahier des charges) : `isLoading` n'est touché que si `startRow === 0`, c'est-à-dire au tout premier bloc. Tous les blocs suivants se chargent silencieusement — les lignes apparaissent quand elles sont prêtes.
- **Deux stratégies de données** :
 - *Cas B (nominal)* : `cacheBlockSize` vaut 10 = la taille de page SWAPI, donc le bloc AG Grid n°N correspond exactement à la page SWAPI n°N+1. Aucune donnée superflue n'est chargée ("pas d'overfetching").
 - *Cas A (recherche/tri)* : on ne peut ni chercher par nom seul ni trier côté serveur, donc on bascule sur le jeu complet (mis en cache), on filtre/trie localement, et on `slice` le bloc demandé.
- `successCallback(rows, count)` : le second argument donne à AG Grid le nombre *total* de lignes, ce qui lui permet de dimensionner la scrollbar correctement dès le début.
- `finalize` garantit que le loader s'éteint dans tous les cas (succès comme erreur).

5.5 Le tri personnalisé

```
// crew/passengers/cargo_capacity are raw SWAPI strings (e.g. "30-165", "1,000,000", "unknown").
const NUMERIC_SORT_COLUMN_IDS = new Set(['pilots.length', 'crew', 'passengers', 'cargo_capacity']);

function parseNumericValue(value: unknown): number | null {
  if (typeof value === 'number') return value;
  if (typeof value !== 'string') return null;
  const match = value.replace(/,/g, '').match(/-?\d+(\.\d+)?/);
  return match ? parseFloat(match[0]) : null;    // "unknown" → null
}

function compareNumeric(a: number | null, b: number | null, direction: number): number {
  if (a === null || b === null) {
    return a === b ? 0 : a === null ? 1 : -1;    // null TOUJOURS en dernier
  }
  return (a - b) * direction;
}

private applySort(items: Starship[], sortModel: SortModelItem[]): Starship[] {
  if (!sortModel.length) return items;

  const { colId, sort } = sortModel[0];
  const direction = sort === 'desc' ? -1 : 1;

  return [...items].sort((a, b) => {
    const aValue = this.getFieldValue(a, colId);
    const bValue = this.getFieldValue(b, colId);

    if (NUMERIC_SORT_COLUMN_IDS.has(colId)) {
      return compareNumeric(parseNumericValue(aValue), parseNumericValue(bValue), direction);
    }
    return String(aValue ?? '').localeCompare(String(bValue ?? '')) * direction;    // tri naturel
  });
}
```

- **Parsing** : on retire les virgules de milliers ("1,000,000" → 1000000) puis on extrait le premier nombre trouvé ("30-165" → 30). Ce qui n'est pas parsable ("unknown", "n/a") devient null .
- **Les null sortent toujours en dernier**, quel que soit le sens du tri : le test `a === null ? 1 : -1` ignore volontairement `direction` . C'était une exigence explicite : "unknown" ne doit jamais se retrouver en tête de liste.
- **Tri naturel** pour les colonnes textuelles : `localeCompare` gère correctement accents et casse.
- `getFieldValue` résout les chemins pointés ('pilots.length') en descendant dans l'objet clé par clé.
- Le tri est déclenché naturellement par AG Grid : cliquer un en-tête modifie `params.sortModel` , la grille purge ses blocs et rappelle `getRows` , qui applique `applySort` .

5.6 La recherche (debounce) et l'édition

Recherche

```
constructor() {
  this.searchInput$.pipe(debounceTime(300), distinctUntilChanged()).subscribe((query) => {
    this.onSearch(query);
  });
}

onSearchInputChange(event: Event): void {
  this.searchInput$.next((event.target as HTMLInputElement).value);
}

onSearch(query: string): void {
  this.searchTerm.set(query);
  this.reachedEnd.set(false);
  this.gridApi?.purgeInfiniteCache(); // force AG Grid à redemander tous les blocs
}
```

Chaque frappe alimente le `Subject`. `debounceTime(300)` attend 300 ms de silence clavier avant de propager (évite une requête par lettre), `distinctUntilChanged` ignore les valeurs identiques. `purgeInfiniteCache()` vide le cache de blocs d'AG Grid : la grille rappelle `getRows`, qui voit le nouveau `searchTerm()` et bascule dans le "cas A".

Édition de cellule

```
onCellValueChanged(event: CellValueChangedEvent<Starship>): void {
  const field = event.colDef.field as keyof Starship | undefined;
  if (event.source !== 'edit' || !field) {
    return; // ignore les changements programmatiques
  }

  const changes = { [field]: event.newValue } as Partial<Starship>;

  this.swapiService.updateStarship(event.data, changes).subscribe({
    error: () => event.node.setDataValue(field, event.oldValue), // rollback si échec
  });
}
```

- Le garde `event.source !== 'edit'` filtre : seules les éditions *utilisateur* déclenchent la sauvegarde (pas les `setDataValue` programmatiques — ce qui éviterait d'ailleurs une boucle infinie avec le rollback).
- Pattern optimiste** : la cellule affiche la nouvelle valeur immédiatement ; si la "sauvegarde" échoue, on restaure l'ancienne valeur.
- La valeur éditée vit **uniquement dans la mémoire de la grille** : pas de backend, pas de localStorage — un rafraîchissement de page restaure la donnée SWAPI d'origine. C'est le comportement demandé par le cahier des charges.

Fin de liste

```
onBodyScrollEnd(): void {
  const total = this.totalCount();
  if (total === null || !this.gridApi) return;
  this.reachedEnd.set(this.gridApi.getLastDisplayedRowIndex() >= total - 1);
}
```

À chaque fin de scroll, on compare l'index de la dernière ligne *affichée* au total : si on voit la dernière ligne, le footer "End of list" apparaît ; si on remonte, il disparaît.

6. Le template HTML

starship-grid.html — 60 lignes, entièrement stylées en Tailwind.

6.1 Structure générale

```
<div class="flex h-dvh w-full flex-col">      <!-- colonne pleine hauteur d'écran -->
  <header>  titre + recherche + compteur      </header>
  <div class="relative flex-1"> loader / erreur / grille </div>  <!-- prend tout l'espace restant -->
  <footer>  "End of list" (conditionnel)      </footer>
</div>
```

`h-dvh` (dynamic viewport height) + `flex-col` : le header et le footer prennent leur hauteur naturelle, la zone grille absorbe tout le reste (`flex-1`). La grille elle-même est en `h-full w-full` dans ce conteneur.

6.2 Le header — recherche et compteur

```
<div class="relative">
  <span class="material-icons pointer-events-none absolute top-1/2 left-2 -translate-y-1/2
    text-base text-gray-400">search</span>

  <input
    type="search"
    placeholder="Search starships by name"
    class="w-64 rounded border border-gray-300 py-1.5 pr-3 pl-8 text-sm
      focus:border-[#405DFF] focus:outline-none"
    (input)="onSearchInputChange($event)"
  />
</div>
@if (searchTerm() && !isLoading() && totalCount() !== null) {
  <span class="text-sm text-gray-500">{{ totalCount() }} result{{ totalCount() === 1 ? '' : 's' }} found</span>
}
```

- L'icône `search` est positionnée en absolu *dans* l'input (conteneur `relative`) ; le `pl-8` de l'input décale le texte pour ne pas chevaucher l'icône ; `pointer-events-none` laisse les clics traverser l'icône jusqu'à l'input.
- `focus:border-[#405DFF]` : valeur arbitraire Tailwind — la bordure passe au bleu `#405DFF` au focus.
- Le compteur n'apparaît que si : une recherche est active, le chargement est fini (pour ne pas montrer un total périmé), et le total est connu. Le pluriel est géré (`1 result` / `2 results`).
- La syntaxe `@if` est le *control flow* natif d'Angular 17+ (remplace `*ngIf`).

6.3 Loader et écran d'erreur

```
@if (isLoading()) {
  <div class="absolute inset-0 z-10 flex items-center justify-center bg-white/60">
    <div class="h-8 w-8 animate-spin rounded-full border-2 border-gray-300 border-t-gray-600"></div>
  </div>
} @else if (hasError()) {
  <div class="absolute inset-0 z-20 flex flex-col items-center justify-center gap-3 bg-white">
    <p>Failed to load starships. Please try again.</p>
    <button (click)="onRetry()">Retry</button>
  </div>
}
```

- Le spinner est un simple `div` rond dont seule la bordure du haut est foncée, animé par `animate-spin` — pas d'image, pas de librairie.

- Les deux overlays couvrent la zone grille (`absolute inset-0`) au-dessus d'elle (`z-10` / `z-20`) ; le fond `bg-white/60` du loader laisse deviner la grille en dessous.

6.4 Les options de la grille

```
<ag-grid-angular
  class="ag-theme-alpine h-full w-full"
  [columnDefs]="colDefs"
  [defaultColDef]="defaultColDef"
  [rowModelType]='infinite'
  [datasource]="dataSource"
  [cacheBlockSize]="cacheBlockSize"      <!-- 10 = taille de page SWAPI -->
  [rowHeight]="42"
  [headerHeight]="48"
  [theme]='legacy'
  [animateRows]="false"
  [enableCellTextSelection]="true"
  (gridReady)="onGridReady($event)"
  (cellValueChanged)="onCellValueChanged($event)"
  (bodyScrollEnd)="onBodyScrollEnd()"
/>
```

Option	Pourquoi
<code>[theme]='legacy'</code>	On importe les CSS de thème historiques (<code>ag-theme-alpine.css</code>) dans <code>styles.css</code> . AG Grid v33+ a une nouvelle Theming API ; mélanger les deux provoque l'erreur #239 — <code>'legacy'</code> dit explicitement "j'utilise les fichiers CSS".
<code>[animateRows]="false"</code>	Désactive l'animation d'apparition des lignes. Corrige un bug rare de lignes invisibles constaté en production et colle mieux à l'exigence "les lignes apparaissent de façon fluide".
<code>[rowHeight]="42"</code> <code>[headerHeight]="48"</code>	Valeurs par défaut du thème Alpine, fixées explicitement pour ne pas dépendre du timing de chargement du CSS (supprime le warning #9 d'AG Grid).
<code>[enableCellTextSelection]="true"</code>	AG Grid bloque la sélection de texte par défaut ; cette option la réactive pour pouvoir copier le contenu des cellules.
<code>(gridReady)</code>	Récupère <code>event.api</code> (le <code>GridApi</code>), nécessaire pour <code>purgeInfiniteCache()</code> et <code>getLastDisplayedRowIndex()</code> .

7. Les styles

7.1 styles.css — la feuille globale

src/styles.css

```
@import "tailwindcss";
@import "@fontsource/inter/400.css";
@import "@fontsource/inter/500.css";
@import "@fontsource/inter/600.css";
@import "@fontsource/inter/700.css";
@import "../node_modules/ag-grid-community/styles/ag-grid.css";
@import "../node_modules/ag-grid-community/styles/ag-theme-alpine.css";
@import "../node_modules/material-icons/iconfont/material-icons.css";

/* Must stay in this entry stylesheet: Tailwind only bakes @theme overrides
   into Preflight's base styles (e.g. body's font-family) when they're part
   of this same compilation pass – a component-scoped stylesheet is compiled
   independently and can't affect it. */
@theme {
  --font-sans: 'Inter', ui-sans-serif, sans-serif;
}
```

- `@import "tailwindcss"` est toute la configuration Tailwind v4 : Preflight (reset), les utilitaires, le thème.
- Les 4 imports `@fontsource/inter` chargent les graisses 400/500/600/700 de la police Inter, **auto-hébergée** (les .woff2 sortent du build, aucun appel à Google Fonts au runtime).
- Les deux CSS AG Grid fournissent la structure et le thème Alpine de la grille (mode "legacy", cf. 6.4).
- `@theme { --font-sans: 'Inter'... }` remplace la police par défaut de Tailwind : tout le document hérite d'Inter.

⚠ Piège — Le commentaire du fichier est crucial : en Tailwind v4, un bloc `@theme` n'affecte les styles de base (le `font-family` du `body`, généré par Preflight) que s'il est compilé **dans le même passage PostCSS** que ces styles de base — donc dans `styles.css`, le fichier listé dans `angular.json`. Déplacer ce bloc dans un CSS de composant (compilé séparément) casserait silencieusement la police globale. Cela a été constaté et vérifié pendant le développement.

7.2 starship-grid.css — les overrides AG Grid

Ce fichier existe parce que **certaines choses sont impossibles en classes Tailwind** : AG Grid génère lui-même son DOM interne (icônes de tri, conteneurs d'en-tête), sur lequel on ne peut pas poser de classes utilitaires depuis les `colDefs`. Combiné à `ViewEncapsulation.None` (chapitre 5.1), ce CSS "sort" du composant pour atteindre ce DOM.

a) La police d'AG Grid

```
.ag-theme-alpine {
  --ag-font-family: 'Inter', ui-sans-serif, sans-serif;
  --ag-header-background-color: #fff;
}
```

Le thème Alpine définit sa propre variable de police, indépendante du `--font-sans` de Tailwind : il faut donc l'aligner séparément. Même principe pour le fond des en-têtes, passé au blanc pour matcher le corps de la grille.

b) Aperçu des flèches de tri au survol

```
/* Non trié → le survol prévisualise la flèche ascendante du prochain clic */
.ag-header-cell-sortable[aria-sort="none"]:hover .ag-sort-ascending-icon.ag-hidden {
  display: inline-flex !important;
  opacity: 0.4;
}
/* Trié ascendant → le survol prévisualise la flèche descendante */
.ag-header-cell-sortable[aria-sort="ascending"]:hover .ag-sort-ascending-icon { display: none !important; }
.ag-header-cell-sortable[aria-sort="ascending"]:hover .ag-sort-descending-icon.ag-hidden {
  display: inline-flex !important;
  opacity: 0.4;
}
/* Trié descendant → le survol prévisualise la disparition du tri */
.ag-header-cell-sortable[aria-sort="descending"]:hover .ag-sort-descending-icon { display: none !important; }
```

AG Grid masque l'icône de tri avec `.ag-hidden { display: none !important; }` tant qu'une colonne n'est pas triée. Ces règles exploitent l'attribut d'accessibilité `aria-sort` posé par AG Grid pour afficher, au survol, une version semi-transparente (`opacity: 0.4`) de l'état *que produira le prochain clic* — un vrai raffinement UX. Le `!important` est obligatoire pour battre celui d'AG Grid.

c) Les icônes Material dans les en-têtes

```
.ag-header-cell-text {
  display: inline-flex;
  align-items: center;
  gap: 4px;
  color: #9ca3af;          /* gris = même couleur que l'icône search du header */
}

.hdr-icon-name .ag-header-cell-text::before,
.hdr-icon-class .ag-header-cell-text::before,
/* ... les 8 colonnes ... */ {
  font-family: 'Material Icons';
  font-size: 16px;
  line-height: 1;
}

.hdr-icon-name .ag-header-cell-text::before { content: 'rocket_launch'; }
.hdr-icon-class .ag-header-cell-text::before { content: 'category'; }
.hdr-icon-model .ag-header-cell-text::before { content: 'widgets'; }
/* manufacturer: 'factory', pilots: 'badge', crew: 'groups',
   passengers: 'airline_seat_recline_normal', cargo: 'inventory_2' */
```

L'astuce : la police Material Icons utilise les **ligatures** — écrire le *nom* de l'icône (`content: 'rocket_launch'`) dans un élément portant cette police fait apparaître le glyphe. Comme il n'existe aucun hook `colDef` pour injecter du HTML dans le texte d'en-tête, chaque colonne reçoit une classe marqueur (`hdr-icon-*` via `headerClass`) et l'icône est insérée en pseudo-élément `::before`. Avantage : le composant d'en-tête natif d'AG Grid (clic-pour-trier, flèches) reste intact.

💡 **Bonne pratique** — Règle de style du projet : **tout style passe par Tailwind** quand c'est techniquement possible (templates, classes des `colDefs`). Le CSS "brut" n'est utilisé que pour le DOM interne d'AG Grid, hors de portée des classes utilitaires — et chaque règle y est commentée pour justifier son existence.

8. Les tests unitaires

11 tests répartis sur 3 fichiers, exécutés par **Vitest** via `npm test`. Aucun test ne touche le vrai réseau : HTTP et service sont mockés.

8.1 swapi.service.spec.ts — pagination et combinaison

Utilise `provideHttpClientTesting()` : le `HttpTestingController` intercepte chaque requête, permet de vérifier l'URL/paramètres, et d'injecter une fausse réponse (`flush`).

Test 1 — le bon paramètre de page

```
it('requests the given page number from the SWAPI starships endpoint', () => {
  service.getStarships(2).subscribe();
  const req = httpMock.expectOne(
    (r) => r.url === 'https://swapi.dev/api/starships/' && r.params.get('page') === '2',
  );
  expect(req.request.method).toBe('GET');
  req.flush(makeResponse({ count: 1 }));
});
```

Test 2 — le cache

```
it('caches a page so the same page is never requested twice', () => {
  service.getStarships(1).subscribe();
  service.getStarships(1).subscribe();
  httpMock.expectOne(() => true).flush(makeResponse({ count: 1 }));
  httpMock.verify(); // ← échouerait s'il restait une 2e requête en attente
});
```

Deux souscriptions, une seule requête HTTP attendue : c'est la preuve du `shareReplay` + Map de cache.

Test 3 — la combinaison des pages

```
it('combines every page into a single, ordered list', () => {
  // page 1 : count=3, 2 vaisseaux → totalPages = ceil(3/2) = 2
  // page 2 : 1 vaisseau
  service.getAllStarships().subscribe((results) => (combined = results));
  httpMock.expectOne((r) => r.params.get('page') === '1').flush(page1);
  httpMock.expectOne((r) => r.params.get('page') === '2').flush(page2);
  expect(combined.map((s) => s.name)).toEqual(['A-wing', 'B-wing', 'X-wing']); // ordre préservé
});
```

8.2 starship-grid.spec.ts — tri et édition

Ici, `SwapiService` entier est remplacé par un mock (`vi.fn()` de Vitest) fourni via `TestBed.configureTestingModule({ providers: [{ provide: SwapiService, useValue: mock }] })`. Les tests appellent **directement** `component.dataSource.getRows(params)` avec de faux `params` — pas besoin de rendre la grille.

Tri numérique + "unknown" en dernier

```
swapiService.getAllStarships.mockReturnValue(of([
  makeStarship({ name: 'Y-wing', crew: '2' }),
  makeStarship({ name: 'Death Star', crew: 'unknown' }),
  makeStarship({ name: 'X-wing', crew: '1' }),
]));

component.dataSource.getRows({ sortModel: [{ colId: 'crew', sort: 'asc' }], ... });

expect(rows.map((s) => s.name)).toEqual(['X-wing', 'Y-wing', 'Death Star']);
// 1 < 2, et "unknown" relégué en dernier malgré le tri ascendant ✓
```

Édition — envoi, rollback, filtrage

- **Envoi** : un événement `source: 'edit'` sur `cargo_capacity` doit appeler `updateStarship(starship, { cargo_capacity: '2000' })`.
- **Rollback** : si le mock renvoie `throwError`, on vérifie `node.setDataValue('cargo_capacity', '1000')` — la cellule est restaurée.
- **Filtrage** : un événement `source: 'api'` (changement programmatique) ne doit *pas* appeler le service.

8.3 app.spec.ts

Deux tests basiques du composant racine : il se crée sans erreur, et le `<h1>` rendu contient bien "Star Wars Fleet".

9. Le déploiement Netlify

Le site est déployé en continu sur <https://swapi-grid.netlify.app> : chaque push sur `main` déclenche un build Netlify.

Le pipeline

1. Netlify clone le dépôt et lit `netlify.toml`.
2. Il installe Node à la version indiquée par `.nvmrc` (cohérence local/CI).
3. Il exécute `npm run build` → `ng build` (production par défaut : minification, tree-shaking, hash des fichiers).
4. Il publie le dossier `dist/swapi-grid/browser`.
5. Le fichier `_redirects` (copié depuis `public/`) installe la règle SPA `/* → /index.html 200`.

Incidents résolus durant le développement (à connaître)

Symptôme	Cause racine	Correctif
Build Netlify échoue : "trying to use <code>tailwindcss</code> directly as a PostCSS plugin"	<code>.postcssrc.json</code> n'avait jamais été commité — Netlify buildait sans la config PostCSS.	Commit du fichier + <code>.nvmrc</code> pour figer Node.
Colonnes visibles mais lignes invisibles (aléatoire, en prod)	Bug de peinture rare lié à l'animation de lignes par défaut d'AG Grid.	<code>[animateRows]="false"</code> .
Warning AG Grid #9 (<code>--ag-row-height</code> non résolu au premier rendu)	La grille se dimensionnait avant que le CSS du thème soit appliqué.	<code>[rowHeight]="42"</code> et <code>[headerHeight]="48"</code> explicites.

i À savoir — SWAPI (`swapi.dev`) est un service gratuit parfois lent ou indisponible. L'application gère ce cas (écran d'erreur + Retry + éviction du cache), mais si le site semble vide, la première hypothèse à vérifier est la disponibilité de l'API elle-même.

10. Concepts clés et pièges à connaître

Synthèse des points non évidents du projet — la liste à relire avant de modifier le code.

10.1 Angular zoneless — toujours des signals

Pas de zone.js : muter une propriété ordinaire depuis un `subscribe` ne re-rend pas le template. Tout état affiché doit être un `signal()`, écrit via `.set()`. (Voir 3.2.)

10.2 Tailwind v4 — CSS-first, et un seul point d'entrée

- Plus de `tailwind.config.js` : la config vit dans le CSS (`@theme`). Le fichier JS a été supprimé du projet.
- Le branchement se fait via `.postcssrc.json` avec le plugin `@tailwindcss/postcss` (et non `tailwindcss` directement — erreur classique v4).
- Le `@theme` qui change la police globale **doit rester dans** `styles.css` (compilation par passage, voir 7.1).
- Préfixe `!` (ex. : `!border-r`) = `!important`, indispensable contre les styles internes d'AG Grid.

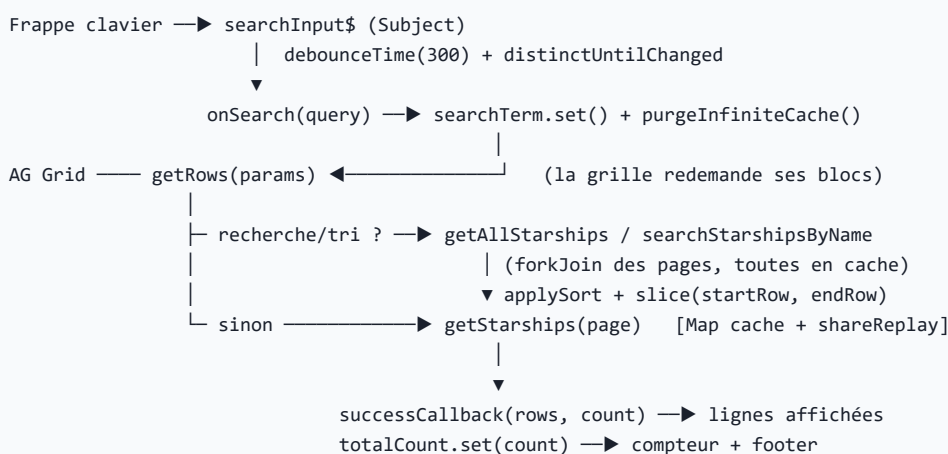
10.3 AG Grid — les règles du jeu de ce projet

- **Thème legacy** : CSS importés + `[theme]='''legacy'''`. Ne jamais mélanger avec la nouvelle Theming API (erreur #239).
- **Infinite Row Model** : la grille "tire" les blocs via `getRows` ; `purgeInfiniteCache()` est la manière de forcer un rechargement (utilisé par la recherche et le Retry).
- `cacheBlockSize` (10) est aligné sur la pagination SWAPI : 1 bloc = 1 page, zéro overfetch.
- `rowNumbers` natif = Enterprise uniquement ; la colonne `#` est un `valueGetter` maison.
- Le DOM interne d'AG Grid n'est pas stylable en Tailwind → CSS dédié + `ViewEncapsulation.None`.

10.4 Les données SWAPI — des chaînes pièges

- Champs numériques = chaînes (`"30-165"`, `"1,000,000"`, `"unknown"`, `"n/a"`) : parser avant tout tri/calcul (`parseNumericValue`).
- Les non-parseables deviennent `null` et sortent **toujours en dernier** au tri, peu importe le sens.
- Le paramètre `?search=` de SWAPI matche aussi `model` : inutilisable pour une recherche par nom seul → filtre local.
- API en lecture seule : toute "écriture" est simulée côté client.

10.5 Les flux RxJS du projet, en une image



10.6 Conventions du projet

- **Style** : Tailwind partout où c'est possible ; CSS brut uniquement pour le DOM interne d'AG Grid, toujours commenté.
- **Nommage git** (emojis) : 🚀 feat, 🐛 fix, 🎨 style, ♻️ refactor, 📄 docs, 🛠️ chore, ✅ test.
- **Fichiers Angular 21 sans suffixe** (`starship-grid.ts`), classe avec suffixe (`StarshipGridComponent`).
- **Textes d'interface en anglais** ("End of list", "results found", "Retry"...).
- TypeScript strict + templates stricts : le compilateur vérifie aussi les bindings HTML.

💡 **Bonne pratique** — Pour explorer le code, le meilleur ordre de lecture est celui de ce document : `main.ts` → `app.config.ts` → `swapi.service.ts` → `starship-grid.ts` → `starship-grid.html` → `starship-grid.css` . Le service et le composant contiennent des commentaires expliquant chaque décision non triviale.